



Universidade Federal do Rio Grande do Norte
Centro de Tecnologia - CT
Departamento de Engenharia Elétrica - DEE
Laboratório de Automação, Controle e Instrumentação - LACI

Processador

ELE3515 - Grupo 01 - Problema 05 - Implementação

Líder:	Rinaldo Tavares da Silva Filho
Redator:	Nicolas Pinheiro Silva
Debatedor:	Eduardo Lira da Silva Filho

Natal, 2 de dezembro de 2025

Resumo

O trabalho a seguir apresenta o desenvolvimento de um processador baseado na arquitetura Harvard, utilizando uma máquina de estados de alto nível para organizar o ciclo de busca, decodificação e execução de instruções. O projeto inclui a implementação dos principais módulos que compõem o processador, como memória de dados, memória de instruções, registrador de instruções (IR), contador de programa (PC), banco de registradores, unidade lógica e aritmética (ULA) e bloco de controle. Foram definidos 16 opcodes, contemplando operações aritméticas, lógicas, de movimentação e salto condicional, os quais estruturam a lógica da execução. O modelo RTL foi desenvolvido a partir de adaptações do exemplo de Vahid (2008), incorporando sinais adicionais para suportar todas as instruções requeridas e garantir o comportamento correto do processador. O resultado é uma arquitetura funcional e didática, capaz de demonstrar de forma clara o fluxo interno de execução e a comunicação entre seus componentes.

Palavras-chave: Arquitetura Harvard; Máquina de Estados; RTL; Processador; ULA; Opcode.

Sumário

1	Introdução	4
2	Fundamentação Teórica	5
3	Desenvolvimento da Solução	7
3.1	Processador	7
3.2	OPCODE	7
3.2.1	Memória de dados	8
3.2.2	Memória de instrução	8
3.2.3	Maquina de estado de alto nível	8
3.3	Projeto RTL	11
3.4	Projeto da Solução	12
3.5	Contador PC	12
3.6	Circuito IR	13
4	implementação	14
4.1	Correção de Projeto	14
4.2	Mux	15
4.3	Demux	15
4.3.1	Banco de Registradores 16x8 bits	16
4.4	Contador PC	16
4.5	Bloco de controle	17
4.5.1	Descrição do codigo 1	18
4.5.2	Descrição do codigo 2	18
5	Conclusão	19
A	Relato semanal	21
A.1	Equipe	21
A.2	Pontos-chaves	21
A.3	Questões de pesquisa	21
A.4	Planejamento da pesquisa	21
A.5	Anexos	22
A.5.1	Codigo 1 - BIN-ASCII	22
A.5.2	Codigo 2 - Multiplicador	23

1 Introdução

O estudo e a implementação de processadores didáticos são fundamentais para a compreensão dos princípios que regem a arquitetura de computadores e o funcionamento interno de sistemas digitais. Este trabalho apresenta o desenvolvimento de um processador baseado na arquitetura Harvard, no qual as memórias de instruções e de dados são separadas a fim de otimizar o fluxo de execução. A construção do projeto envolve a modelagem de componentes essenciais, como a Unidade Lógica e Aritmética (ULA), o banco de registradores, o contador de programa (PC), o registrador de instruções (IR) e o bloco de controle, além da definição do conjunto de instruções (opcode) e da elaboração de uma máquina de estados de alto nível. A partir de uma abordagem orientada ao projeto RTL, o trabalho busca integrar teoria e prática, permitindo uma visão aprofundada da organização e operação interna de processadores simples. Dessa forma, o relatório apresenta os fundamentos teóricos, o desenvolvimento da solução e a análise funcional dos módulos que compõem o sistema.

2 Fundamentação Teórica

O componente principal deste trabalho é o processador, ele é formado por muitos componentes seus principais são a unidade logica aritmetica, a unidade de ponto flutuante, registros , e o cache.

A ula é responsável por realizar todas as operações aritméticas e lógicas nos dados,sobre ela SHARMA, 2022 traz a seguinte analise:

"A ULA (Unidade Lógica e Aritmética) é o principal componente de um processador, responsável por executar diversas operações aritméticas e lógicas. Trata-se de um circuito integrado dentro da CPU/GPU, razão pela qual também é conhecida como unidade de inteiros (UI). Este é o último componente a realizar cálculos no processador."

A fpu é basicamente um coprocessador, ela trabalha calculando numeros em ponto flutuante, sobre ela SHARMA, 2022 tem a seguinte explicação:

"Faz parte do sistema computacional utilizado para realizar operações com números de ponto flutuante. Essas operações incluem raiz quadrada, multiplicação, divisão, subtração e adição. Pode executar funções transcendentais, como funções trigonométricas e exponenciais; no entanto, pode não ser precisa."

Os registradores são um local de acesso rápido disponível para o processador SHARMA, 2022 explica da seguinte forma:

"Os registradores são tipos de memória do computador que aceitam, transferem e armazenam dados e instruções em uso. Eles instruem a ULA (Unidade Lógica e Aritmética) sobre os processos que devem ser executados e armazenam os resultados dessas operações. "

O cache é uma memória interna ao processador usada para armazenamento temporário de dados, A cache é uma memória menor, porém mais rápida, localizada próxima ao núcleo do processador. Essa memória armazena cópias dos dados das áreas principais frequentemente acessadas. Existem basicamente três níveis de cache: L1, L2 e L3. (SHARMA, 2022).

Os processadores possuem basicamente duas arquiteturas Von Neumann e Harvard. A arquitetura de von Neumann é amplamente utilizada na maioria dos computadores ela seguiu cronologicamente a arquitetura de Harvard, que separava as unidades de memória e processamento. O projeto de Von Neumann aprimorou significativamente o desempenho do computador, armazenando e executando instruções de forma eficiente. Essa arquitetura unifica a unidade de processamento, a memória e os dispositivos de entrada e saída por meio de um único barramento de sistema.(ASHTARI, 2024)

Na Arquitetura Harvard, teoricamente com ela o microcontrolador pode trabalhar mais rápido, ela emprega unidades de memória e barramentos individuais para cada uma, o que pode levar a uma execução de instruções mais rápida e a um desempenho

aprimorado em determinadas aplicações. Essa arquitetura é usada para projetar processadores e microcontroladores modernos, principalmente quando fatores como velocidade e eficiência são importantes.(ASHTARI, 2024).

Uma linguagem de programação que tem haver com o assunto é a linguagem assembly (ou linguagem de montagem), frequentemente abreviada como asm, é uma linguagem de programação de baixo nível para um computador ou outro dispositivo programável, na qual existe uma correspondência muito forte (geralmente um para um) entre a linguagem e as instruções de código de máquina da arquitetura. Cada linguagem assembly é específica para uma determinada arquitetura de computador. Em contraste, a maioria das linguagens de programação de alto nível são geralmente portáteis entre múltiplas arquiteturas, mas requerem interpretação ou compilação. (Archer 2016)

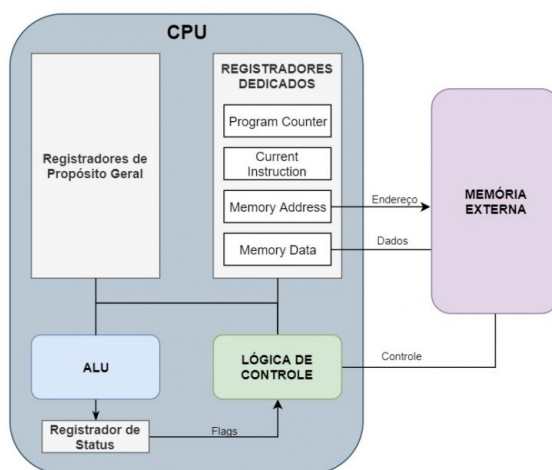


Figura 2.1: funcionamento de um processador (Archer 2016)

Baseado na imagem 1.1 o processador funciona repetindo um ciclo de busca, decodificação e execução. Primeiro, o Program Counter envia o endereço da próxima instrução para o MAR, e a memória devolve essa instrução para o MDR, que a repassa ao registrador de instrução. A Lógica de Controle decodifica o comando e já incrementa o PC. Dependendo do tipo da instrução, o processador pode realizar operações na ALU, ler dados da memória ou escrever dados nela. Todos esses passos envolvem troca de informações entre os registradores dedicados, a ALU, a Lógica de Controle e a memória externa. (“Arquitetura de um microprocessador - Aula 2 - MC - Mundo Projetado”, 2020)

3 Desenvolvimento da Solução

3.1 Processador

O processador desenvolvido neste trabalho tem como base a arquitetura Harvard, com a memória de instruções separada da memória de dados. Foi utilizado como referência o processador descrito por (Vahid 2008). Nesse modelo, são utilizadas uma memória de instruções, uma memória de dados, um registrador de instruções (IR), um contador de programa (PC), o bloco de controle, um banco de registradores, uma ULA e um multiplexador.

O processador é dividido em um bloco de controle, que se comunica com a memória de instruções, e um bloco operacional, que se comunica com a memória de dados, seguindo o padrão RTL. A máquina de estado pode ser simplificada para três tarefas: busca, decodificação e execução.

No estado de busca, o processador realiza a obtenção da próxima instrução a ser executada. Nesse estágio, o PC fornece o endereço da instrução que deve ser lido na memória de instruções. O conteúdo armazenado neste endereço é, então, transferido para o IR, que mantém a instrução por um clock.

No estado de decodificação, o processador interpreta o conteúdo armazenado no registrador de instruções. O bloco de controle analisa o código de operação (opcode), que neste caso é de 4 bits, e determina quais sinais de controle devem ser ativados para que a instrução seja corretamente executada no estado seguinte.

No estado de execução, são realizadas as operações propriamente ditas especificadas pela instrução. O comportamento desse estado depende diretamente do tipo da instrução previamente decodificada; no caso deste trabalho, há 16 operações diferentes.

3.2 OPCODE

Os opcode representam as instruções que o processador pode executar. No modelo deste projeto, cada instrução possui 4 bits, permitindo até 16 instruções diferentes, que incluem as operações aritméticas, lógicas, de movimentação de dados e de pulso de instruções. Cada opcode tem uma representação literal; estes estão na Tab. 3.1, que apresenta os opcodes e suas respectivas representações em binário.

Tabela 3.1: Tabela de opcode e sua representação binária

0000	0001	0010	0011	0100	0101	0110	0111	1000	1001	1010	1011	1100	1101	1111
LDR	STR	MOV	ADD	SUB	AND	OR	NOT	XOR	CMP	JMP	JNC	JC	JNZ	JZ

3.2.1 Memória de dados

Na arquitetura Harvard, a memória de dados é fisicamente separada da memória de instruções. Ela é utilizada para armazenar valores que serão manipulados durante a execução do programa, como variáveis temporárias e resultados intermediários. No projeto implementado, a memória de dados possui endereços de 8 bits, permitindo até 256 posições de armazenamento de 8 bits cada.

3.2.2 Memória de instrução

A memória de instruções projetada possui 256 índices de 16 bits cada, onde cada posição armazena uma instrução completa, incluindo o opcode e os operandos. Esse tamanho permite armazenar até 256 comandos diferentes, que é suficiente para implementar os códigos propostos.

O projeto das memórias de dados e das memórias de instruções pode ser implementado seguindo o mesmo modelo disponível para o tipo de memória implementada. No projeto de número 4, a fifo.

3.2.3 Máquina de estado de alto nível

A máquina de estado de alto nível organiza a execução do processador em três fases principais: busca, decodificação e execução, sendo a execução dividida em 16 estados. Cada estado controla os sinais do bloco de controle, direcionando o fluxo de dados entre registradores, ULA e memórias.

A forma como a máquina de estados de alto nível foi desenvolvida gerou um grande número de estados, pois cada função da etapa de execução foi implementada como um estado independente. Assim, torna-se inviável representar graficamente todas as operações em um único diagrama de estados devido à quantidade de transições, conforme Fig. 3.1. Por esse motivo, foi elaborada uma tabela que organiza e descreve cada estado operacional, permitindo uma compreensão das ações realizadas em cada etapa do processamento.

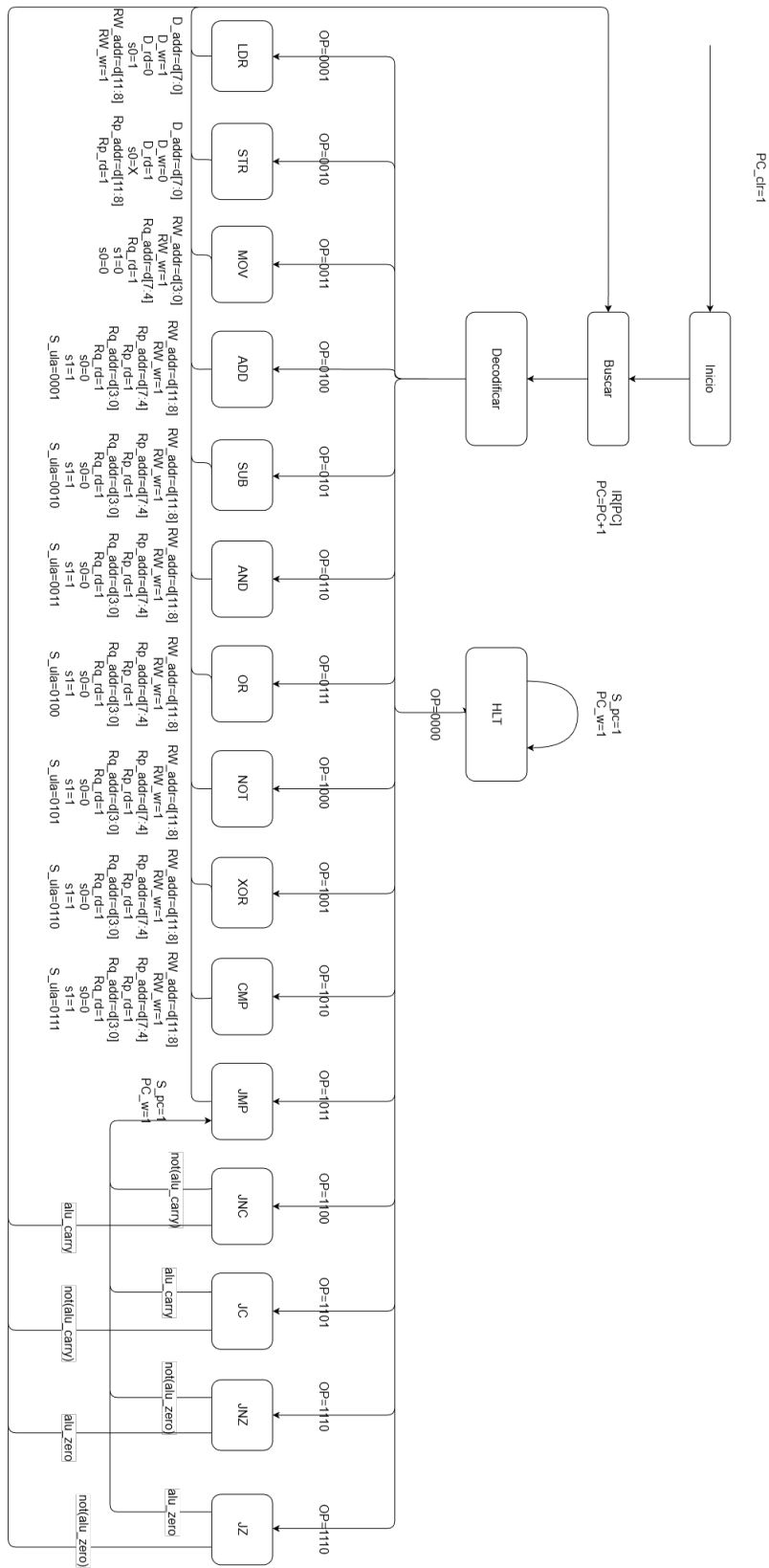


Figura 3.1: Máquina de estado de alto nível

No estado de início, o contador PC é reiniciado por meio do sinal PC_clr . Logo

após, inicia-se o loop principal, que, a cada iteração, executa uma operação, seja aritmética, de movimentação de dados ou de pulo de instrução, cada uma delas necessita de diferentes saídas pelo bloco de controle para que suas específicas funções operem de forma esperada, conforme ilustrado na Fig. 3.1.

O loop só é interrompido quando o opcode 0000 é detectado, indicando a instrução HLT, isto é, a finalização do programa. Nesse caso, o processador permanece preso nesse estado, pois nesse estado são acionados os sinais S_pc e PC_w, que, quando HIGH, gravam o contador PC com o valor de saída menos uma unidade; dessa forma, o contador nunca sai do índice do opcode 0000.

Apos o estado de Decodificação, é passado para um dos 16 estados de execução, sendo decidido por meio do opcode. as saídas do bloco de controle variam em cada estado, seguindo a Tab.3.2.

Tabela 3.2: Tabela de Sinais de Controle ULA e Dados

Inst.	D_addr	D_wr	D_rd	s0	s1	RW_addr	RW_wr	Rp_addr	Rq_addr	Rp_rd	Rq_rd	S_ula
LDR	d[7:0]	1	0	1	0	d[11:8]	1	—	—	—	—	—
STR	d[7:0]	0	1	0	1	—	—	—	d[11:8]	0	1	—
MOV	—	—	—	0	0	d[3:0]	1	—	d[7:4]	0	1	—
ADD	—	—	—	0	1	d[11:8]	1	d[7:4]	d[3:0]	1	1	0001
SUB	—	—	—	0	1	d[11:8]	1	d[7:4]	d[3:0]	1	1	0010
AND	—	—	—	0	1	d[11:8]	1	d[7:4]	d[3:0]	1	1	0011
OR	—	—	—	0	1	d[11:8]	1	d[7:4]	d[3:0]	1	1	0100
NOT	—	—	—	0	1	d[11:8]	1	d[7:4]	d[3:0]	1	1	0101
XOR	—	—	—	0	1	d[11:8]	1	d[7:4]	d[3:0]	1	1	0110
CMP	—	—	—	0	1	d[11:8]	1	d[7:4]	d[3:0]	1	1	0111

Os estados de salto dependem das entradas alu_carry e alu_zero, que correspondem às saídas de carry e zero da ULA. Dessa forma, eles se caracterizam como uma máquina de Mealy, já que a transição de estado e as saídas dependem tanto do estado atual quanto das entradas. Os estados de salto condicionais, ou seja, aqueles que dependem de uma condição específica, estão detalhados na Tabela 3.3.

Tabela 3.3: Tabela de Sinais de Controle ULA e Dados

Inst.	S_pc	PC_w	alu_carry	alu_zero	próximo_estado
JMP	1	1	—	—	Início
JNC	0	0	0	—	JMP
JNC	0	0	1	—	Início
JC	0	0	1	—	JMP
JC	0	0	0	—	Início
JNZ	0	0	—	0	JMP
JNZ	0	0	—	1	Início
JZ	0	0	—	1	JMP
JZ	0	0	—	0	Início

3.3 Projeto RTL

O projeto RTL segue o exemplo descrito por (Vahid 2008), na página 454, conforme ilustrado na Fig. 3.2, com leves adaptações para atender a todas as operações necessárias. Entre essas adaptações estão o incremento dos sinais $S_ula[3:0]$, permitindo comportar todas as operações requeridas, e a adição do sinal de carry, denominado ula_carry .

Os comandos recebidos pelo bloco de controle são provenientes da instrução: o opcode, os 12 bits de dados, além dos sinais ula_zero e ula_carry , referentes aos flags de zero e carry gerados pela ULA. Os sinais de saída são D_addr , D_rd e D_wr , que correspondem, respectivamente, ao endereço onde ocorrerá a leitura ou escrita, ao enable de leitura e ao enable de escrita. De forma similar, no bloco de Registradores existe um endereço para escrita e dois para leitura, cada um com seus respectivos sinais de habilitação. Além disso há o sinal S_ula que controla qual operação aritmética a ULA irá operar, o sinal $s0$ que multiplexa qual sinal irá ser gravado nos Registradores.

Já na parte do bloco de controle tem os sinais PC_clr que é utilizado para zerar o PC, o sinal PC_w que quando ativo o valor do contador é modificado para valor da sua entrada a direita, é o sinal S_pc que multiplexa se o PC vai receber o seu valor decrescido de um, ou se vai receber o valor que vem do IR.

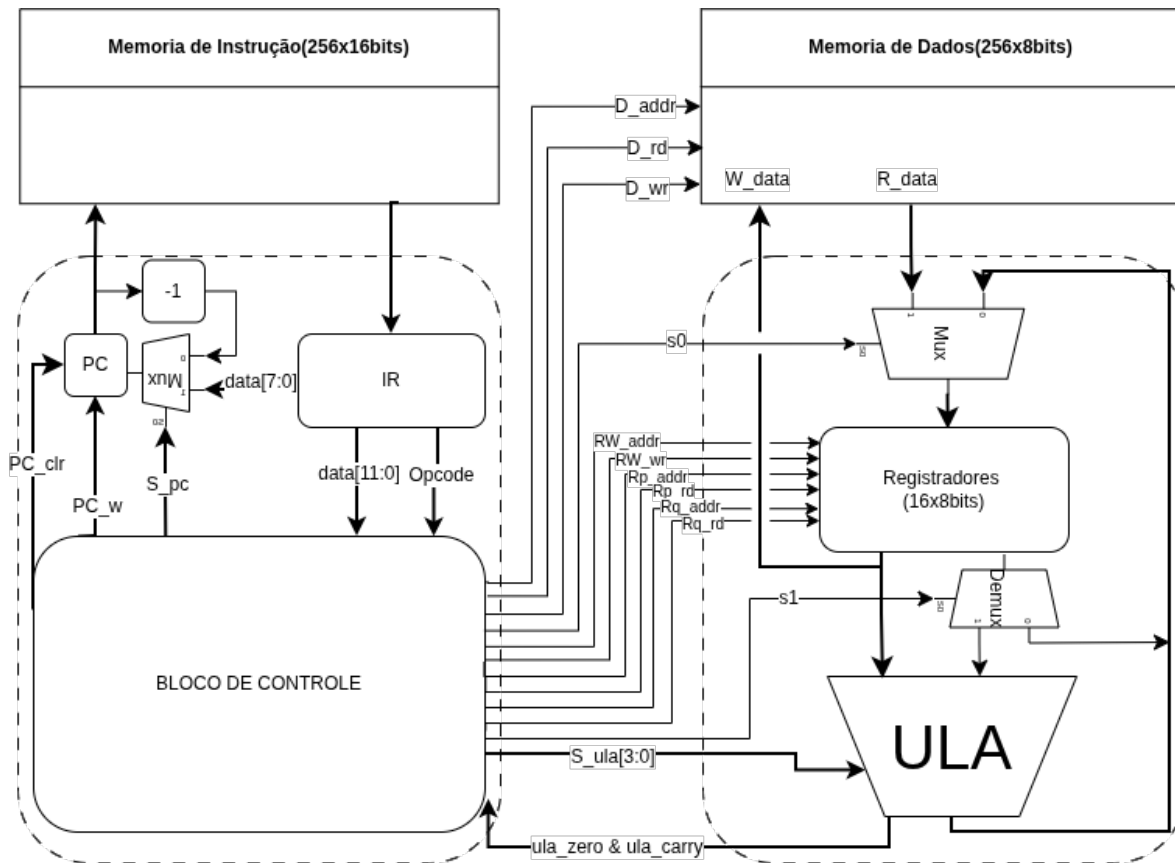


Figura 3.2: Projeto RTL

3.4 Projeto da Solução

O registrador de 16x8bits, é um registrador um pouco diferente já que tem 2 endereços de leitura e um de gravação, além dos seus devidos habilitadores, no entanto é possível construir ele usando um registrador convencional. Para implementar a funcionalidade do segundo endereço de leitura foi adicionado ao circuito outro multiplexador de saída, conforme Fig: 3.3. O resto do circuito permanece com a mesma funcionalidade.

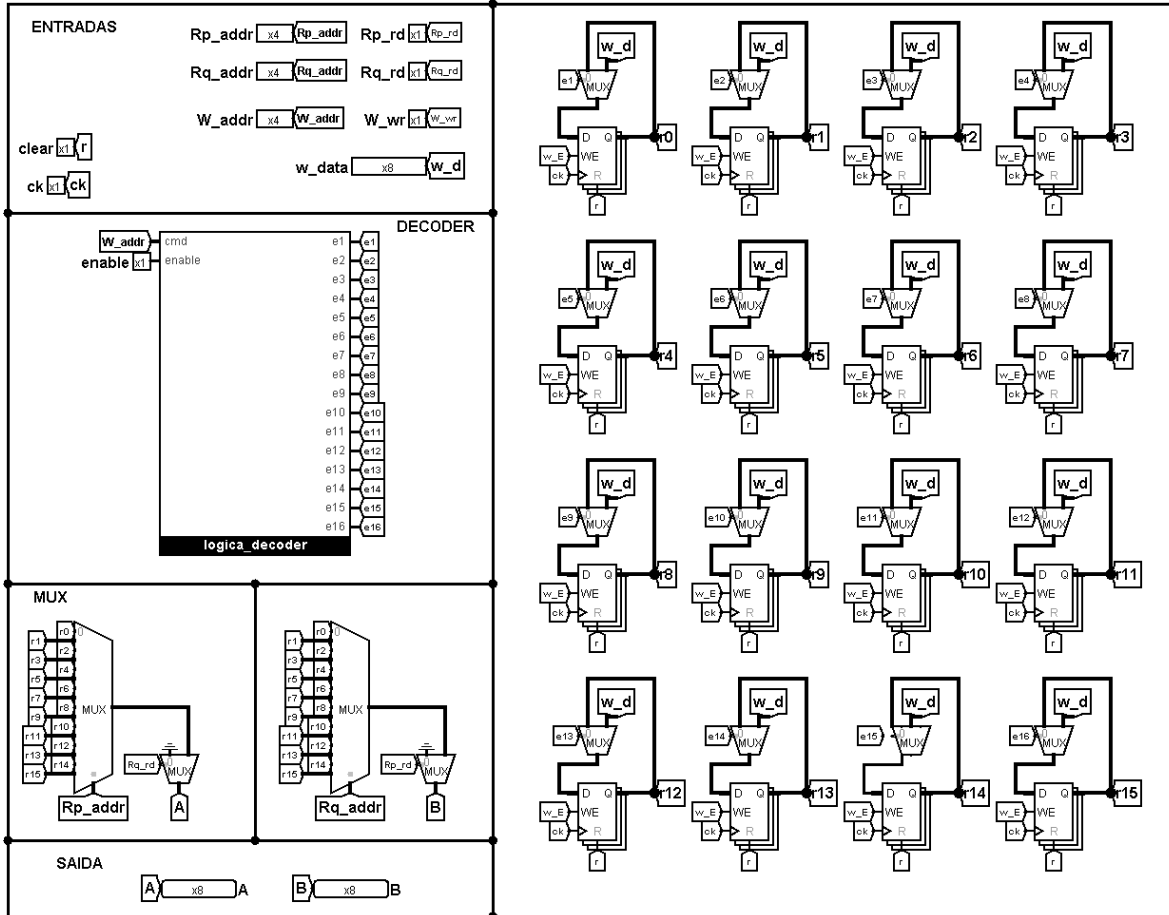


Figura 3.3: Registrador 16x8bits

3.5 Contador PC

O contador de programa (PC) é um contador simples, conforme ilustrado na Fig. 4.5. Para permitir o carregamento de um valor específico no PC, foi adicionado um multiplexador que possibilita sobrescrever o valor armazenado no registrador. Essa funcionalidade é controlada pelo sinal V_enable : quando este sinal está em nível lógico alto, o valor presente na entrada V_sn é carregado no PC.

O reset do contador é assíncrono e é realizado diretamente pelo registrador.

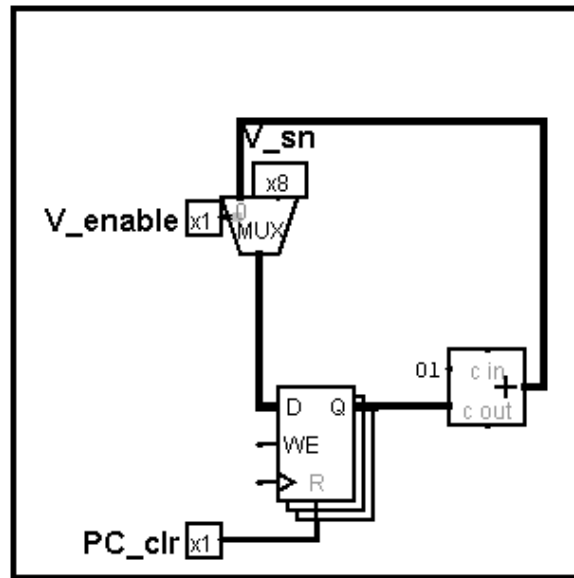


Figura 3.4: Contador PC

3.6 Circuito IR

O circuito do registrador de instruções consiste em um registrador de 16 bits, estruturado de forma a fornecer quatro saídas de 4 bits cada. Os 4 bits mais significativos correspondem ao opcode da instrução, enquanto os demais grupos de 4 bits representam os operandos, normalmente utilizados como endereços ou campos que indicam onde a operação será realizada.

4 implementação

4.1 Correção de Projeto

O demultiplexador originalmente utilizado para selecionar se a entrada do registrador seria proveniente da ULA ou do próprio registrador foi substituído por um multiplexador. Essa alteração foi necessária para evitar que os sinais de saída da ULA fossem combinados com os do demux, o que poderia resultar em comportamentos indeterminados no valor carregado pelo registrador. Com essa modificação, o projeto RTL passa a corresponder ao mostrado na Fig. 4.1.

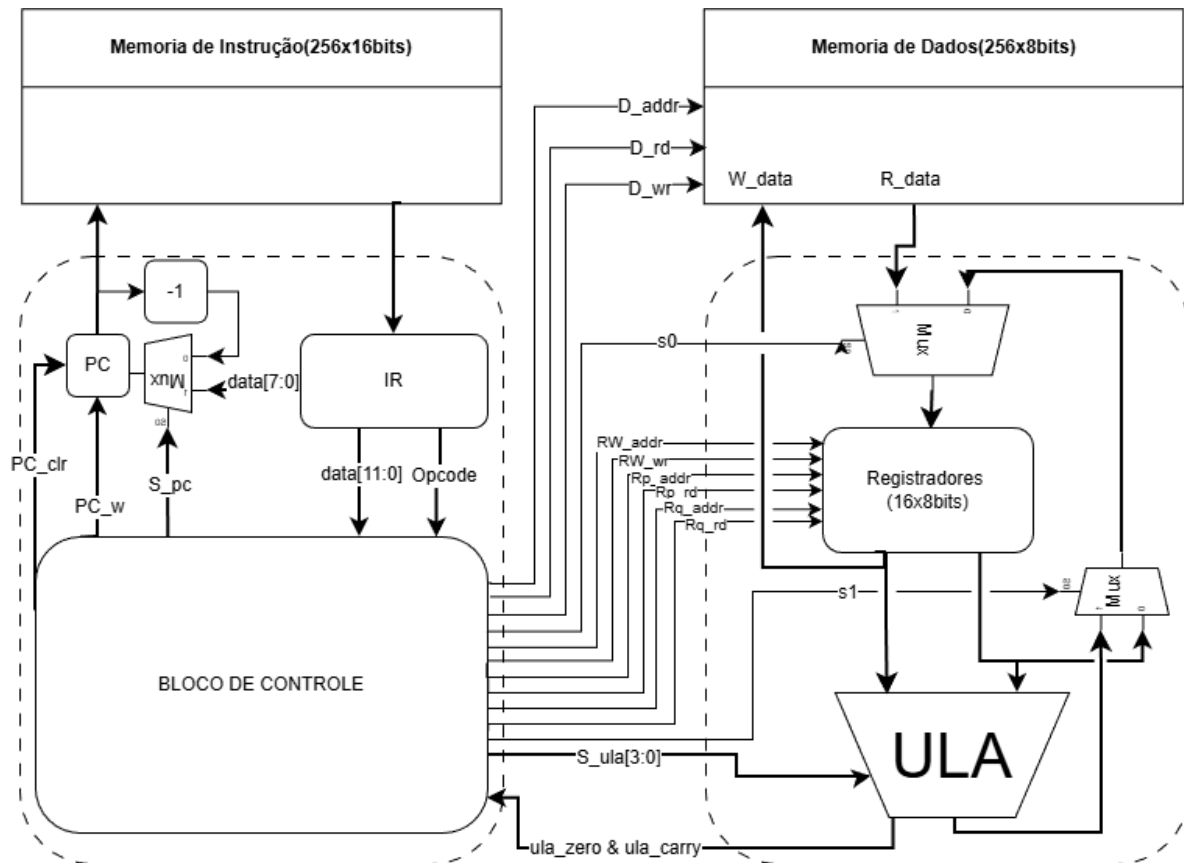


Figura 4.1: Projeto RTL Corrigido

4.2 Mux

O multiplexador foi bem simples; basicamente, foi preciso usar apenas portas lógicas, construindo um mux de 1 bit e depois vetorizar para trabalhar com 8 bits. Também foi feita uma alteração no sinal de seleção para evitar um erro. a função dele é receber duas entradas e escolher qual delas vai para a saída a partir do bit de seleção.



Figura 4.2: resultado do mux 2x1 8 bits

4.3 Demux

O demux foi tão simples quanto o mux; também foi preciso apenas algumas portas lógicas, montando um demux de um bit e, depois, vetorizando para 8, fazendo também o ajuste no S. ele faz exatamente o contrario do mux: ele tem uma entrada e duas saídas, e o bit de seleção diz para qual das duas vai a informação da entrada.



Figura 4.3: resultado do demux 2x1 8 bits

4.3.1 Banco de Registradores 16x8 bits

O banco de registradores segue a mesma estrutura apresentada anteriormente no projeto da LIFO, utilizando um decodificador para habilitar apenas o registrador selecionado para escrita e multiplexadores para selecionar os registradores lidos. O módulo é composto por 16 registradores de 8 bits, um decodificador 4×16 responsável por gerar os sinais de habilitação de escrita e dois multiplexadores 16×1 que permitem a leitura simultânea de dois registradores distintos.

A escrita ocorre apenas quando `wr_reg` está habilitado, e o decodificador ativa exclusivamente o registrador cujo endereço corresponde a `wr_addr`. Para leitura, os multiplexadores retornam o conteúdo indicado por `rp_addr` e `rq_addr`, sendo que as saídas só são disponibilizadas quando os sinais `rp_reg` ou `rq_reg` estão em nível alto. Assim, o funcionamento permanece idêntico ao modelo já discutido no projeto da LIFO, diferenciando-se apenas pelo tamanho do vetor e pela presença de duas portas de leitura.

A simulação do registrador pode ser visto na Fig. 4.4, que mostra o funcionamento, primeiro é gravado valores em diferentes endereços, logo após é feita a leitura com ambas saídas.

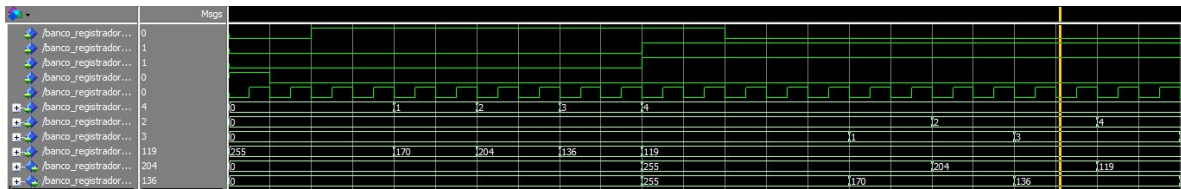
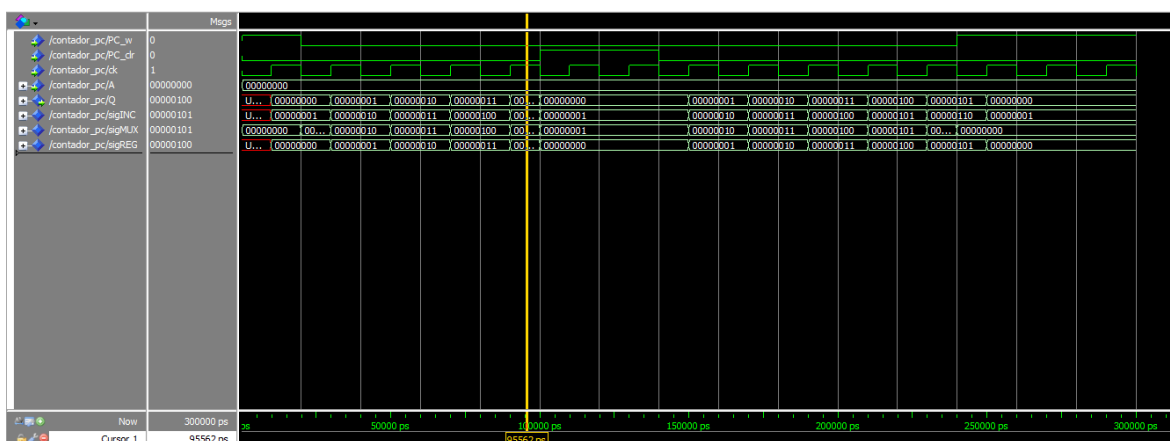


Figura 4.4: Simulação registrador 16x8bits

4.4 Contador PC

A implementação do contador do processador foi realizada a partir de um registrador de 8 bits, dispensando o uso do sinal de *enable*. Isso se deve ao fato de que a seleção do valor a ser armazenado é feita por meio de um multiplexador conectado à entrada do registrador. O sinal seletor desse multiplexador é fornecido pelo bloco de controle do processador, que decide se o contador deve continuar incrementando ou permanecer no valor atual. Além disso, o contador dispõe de uma entrada *clear*, responsável por zerar o conteúdo do registrador.

Na arquitetura do contador, também foi empregado um incrementador de 8 bits em série com a saída do registrador. Essa saída, que representa o valor atual do contador, alimenta tanto uma das entradas do multiplexador quanto o próprio barramento de saída do contador.



4.5 Bloco de controle

A implementação do bloco de controle do processador foi realizada utilizando uma máquina de estados finitos (FSM), responsável pela definição dos sinais de controle de cada estágio de execução da instrução. Para isso, foram declaradas duas variáveis internas do tipo `state_type`, denominadas `y_present` e `y_next`, que armazenam o estado atual e o próximo estado, respectivamente, como pode ser visto na Fig. 4.6.

Os estados definidos representam todas as fases de operação do processador, sendo eles: INIC, BUSC, DECO, DLDR, DSTR, DMOV, DADD, DSUB, DAND, DOR, DNOT, DXOR, DCOMP, DJMP, DJNC, DJC, DJNZ, DJZ e HALT, correspondendo às instruções implementadas no conjunto ISA do projeto.

A transição entre os estados ocorre a partir do conteúdo da instrução localizada no barramento `data` (15 down to 12), que define o opcode executado. No estado `DECO`, é feita a decodificação da instrução, direcionando para o estado operacional apropriado. Cada estado subsequente é responsável por ativar exclusivamente os sinais de controle necessários para execução da instrução em questão, como habilitação de registradores, seleção de multiplexadores, operações da ULA e escrita/leitura de memória.

As instruções aritméticas e lógicas utilizam o sinal `S_ula` para selecionar a operação da ULA, enquanto os saltos condicionais utilizam os sinais `alu_zero` e `alu_carry` para determinar se ocorre mudança no fluxo de execução ou retorno ao estado de busca de instrução. O estado `HALT` mantém o processador parado indefinidamente.

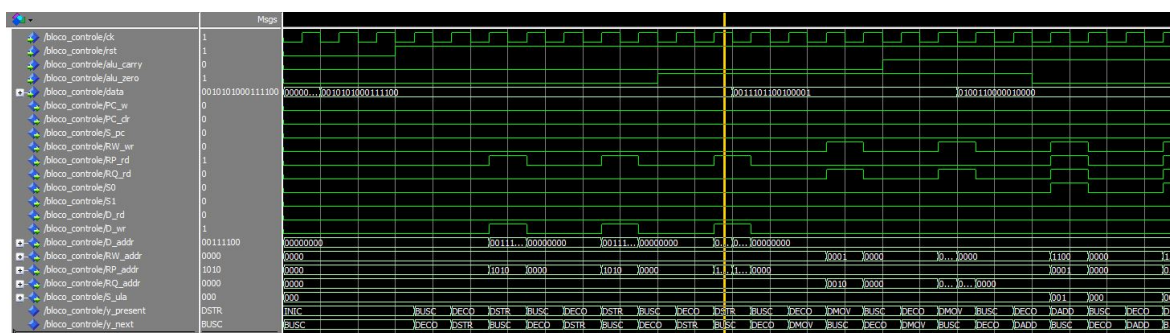


Figura 4.6: Simulação bloco de controle

4.5.1 Descrição do código 1

O código realiza a conversão de um valor inteiro de 0 a 255 em sua representação decimal por meio de operações de subtrações sucessivas.

1. **Inicialização:** São carregadas constantes nos registradores:
 - R1 contém o valor de entrada,
 - R2, R3 e R4 armazenam os divisores 100, 10 e 1,
 - R5 contém o valor 48, usado para converter números para ASCII.
2. **Cálculo das centenas:** O programa compara repetidamente o valor de entrada com 100. Enquanto o valor é maior ou igual, subtrai 100 e incrementa um contador (R6). O resultado final é convertido para ASCII e salvo na memória.
3. **Cálculo das dezenas:** O mesmo procedimento é repetido utilizando o valor 10 como divisor. O contador de dezenas também é convertido para ASCII e armazenado.
4. **Cálculo das unidades:** O valor restante em R1 corresponde às unidades, sendo convertido diretamente para ASCII e salvo.
5. **Encerramento:** O programa finaliza a execução com a instrução HLT.

Em resumo, o algoritmo decompõe o número usando subtrações sucessivas e converte cada dígito para ASCII antes de armazená-lo na memória.

4.5.2 Descrição do código 2

O algoritmo realiza a multiplicação entre dois números de 8 bits por meio de somas sucessivas. Inicialmente, são carregados nos registradores o multiplicando (R1), o multiplicador (R2) e o valor constante 1 (R3), além da inicialização do resultado de 16 bits nos registradores R5:R4.

O laço principal repete a operação de soma enquanto o valor do multiplicador for diferente de zero. Em cada iteração, o multiplicando é somado ao registrador R4. Caso ocorra transbordo (carry), o byte do resultado (R5) é incrementado. Em seguida, o contador (R2) é decrementado, caracterizando o processo de multiplicação por meio de adições repetidas.

Quando o multiplicador atinge o valor zero, o laço é encerrado e o resultado final de 16 bits é armazenado na memória. O programa então finaliza sua execução com a instrução HLT.

5 Conclusão

O desenvolvimento do processador apresentado neste trabalho permitiu compreender, de maneira prática e estruturada, como os elementos fundamentais de uma arquitetura digital interagem para formar um sistema funcional. A partir da definição da arquitetura Harvard, da criação do conjunto de instruções e da modelagem da máquina de estados, foi possível teorizar e, agora, implementar um processador capaz de executar operações aritméticas, lógicas, de movimentação e de salto. O projeto RTL forneceu uma visão clara do fluxo de dados e sinais de controle, reforçando a importância da organização modular e da precisão na definição dos caminhos internos. A construção dos módulos — como o PC, o IR, o banco de registradores e as memórias — evidenciou a relevância de cada componente no ciclo de busca, decodificação e execução. Assim, o trabalho contribuiu significativamente para o entendimento da arquitetura de processadores e consolidou a aplicação prática dos conceitos estudados em sala.

Referências Bibliográficas

Archer 2016 ARCHER, B. *Assembly Language For Students*. North Charleston, SC, USA: CreateSpace Independent Publishing Platform, 2016. ISBN 1540370712.

Vahid 2008 VAHID, F. *Sistemas digitais: projeto, otimização e HDLs*. Porto Alegre: Artmed, 2008. 560p; 25 cm. ISBN 978-85-7780-190-9.

SHARMA, J. What is Processor: Components and Types - Shiksha Online. Disponível em: <<https://www.shiksha.com/online-courses/articles/what-is-processor/>>.

ASHTARI, H. Von Neumann Architecture vs. Harvard Architecture | Spiceworks. Disponível em: <<https://www.spiceworks.com/tech/tech-general/articles/von-neumann-architecture-vs-harvard-architecture/>>.

Arquitetura de um microprocessador - Aula 2 - MC - Mundo Projetado. Disponível em: <<https://mundoprojetado.com.br/arquitetura-de-um-microprocessador/>>.

A Relato semanal

A.1 Equipe

Função no grupo	Nome completo do aluno	Responsável por
Líder	Rinaldo Tavares da Silva Filho	Função: Relatório, Projeto do Bloco de controle em VHDL, adaptação da ULA do projeto 1 e montagem final do código.
Redator	Nicolas Pinheiro Silva	Funções: Relatório, Código VHDL do PC e IC.
Debatedor	Eduardo Lira da Silva Filho	Funções: Relatório, Código VHDL do MUX, DEMUX e bloco de registrador 16x8 bits.

A.2 Pontos-chaves

Projeto RTL, maquina de estados de alto nível;

A.3 Questões de pesquisa

Projetos RTL, maquina de estados de alto nível;

A.4 Planejamento da pesquisa

([Vahid 2008](#)) apenas.

A.5 Anexos

A.5.1 Código 1 - BIN-ASCII

```
// Inicialização
LDR R0, 11111110b      // R0 = 0
LDR R1, 00000001b      // Entrada
LDR R2, 11111010b      // 100
LDR R3, 11111011b      // 10
LDR R4, 11111100b      // 1
LDR R5, 11111101b      // 48 (ASCII offset)
// --- Calcular Centenas ---
MOV R6, R0

LOOP_100:
    CMP R7, R1, R2
    JC FIM_100
    SUB R1, R1, R2
    ADD R6, R6, R4
    JMP LOOP_100

FIM_100:
    ADD R6, R6, R5
    STR R6, 00010000b    // endereço 0x10
// --- Calcular Dezenas ---
MOV R6, R0

LOOP_10:
    CMP R7, R1, R3
    JC FIM_10
    SUB R1, R1, R3
    ADD R6, R6, R4
    JMP LOOP_10

FIM_10:
    ADD R6, R6, R5
    STR R6, 00010001b    // endereço 0x11
// --- Calcular Unidade ---
ADD R1, R1, R5
STR R1, 00010010b      // endereço 0x12

HLT
// ---      Fim      --- //
```

A.5.2 Código 2 - Multiplicador

```
// Inicialização
LDR R0, 11111110b    // R0 = 0 (Zero)
LDR R1, 00000001b    // R1 = Multiplicando (A)
LDR R2, 00000010b    // R2 = Multiplicador (B)
LDR R3, 11111100b    // R3 = 1 (Constante)
// Registradores de Resultado
MOV R4, R0            // R4 = Resultado Low
MOV R5, R0
// R5 = Resultado High
LOOP_MULT:
    // Verifica se o Multiplicador é 0
    CMP R6, R2, R0
    JZ FIM_MULT
    // Soma: Res = Res + A
    ADD R4, R4, R1
    // Verifica Carry do byte baixo
    JNC NO_CARRY
    ADD R5, R5, R3     // Incrementa o byte alto

NO_CARRY:
    // Decrementa o contador
    SUB R2, R2, R3
    JMP LOOP_MULT

FIM_MULT:
    // Armazena o resultado
    STR R5, 00100000b  // Endereço 0x20
    STR R4, 00100001b  // Endereço 0x21

HLT
```